

Saving and Loading Models

In this notebook, I'll show you how to save and load models with PyTorch. This is important because you'll often want to load previously trained models to use in making predictions or to continue training on new data.

```
In [1]: %matplotlib inline
%config InlineBackend.figure_format = 'retina'

import matplotlib.pyplot as plt

import torch
from torch import nn
from torch import optim
import torch.nn.functional as F
from torchvision import datasets, transforms

import helper
import fc_model

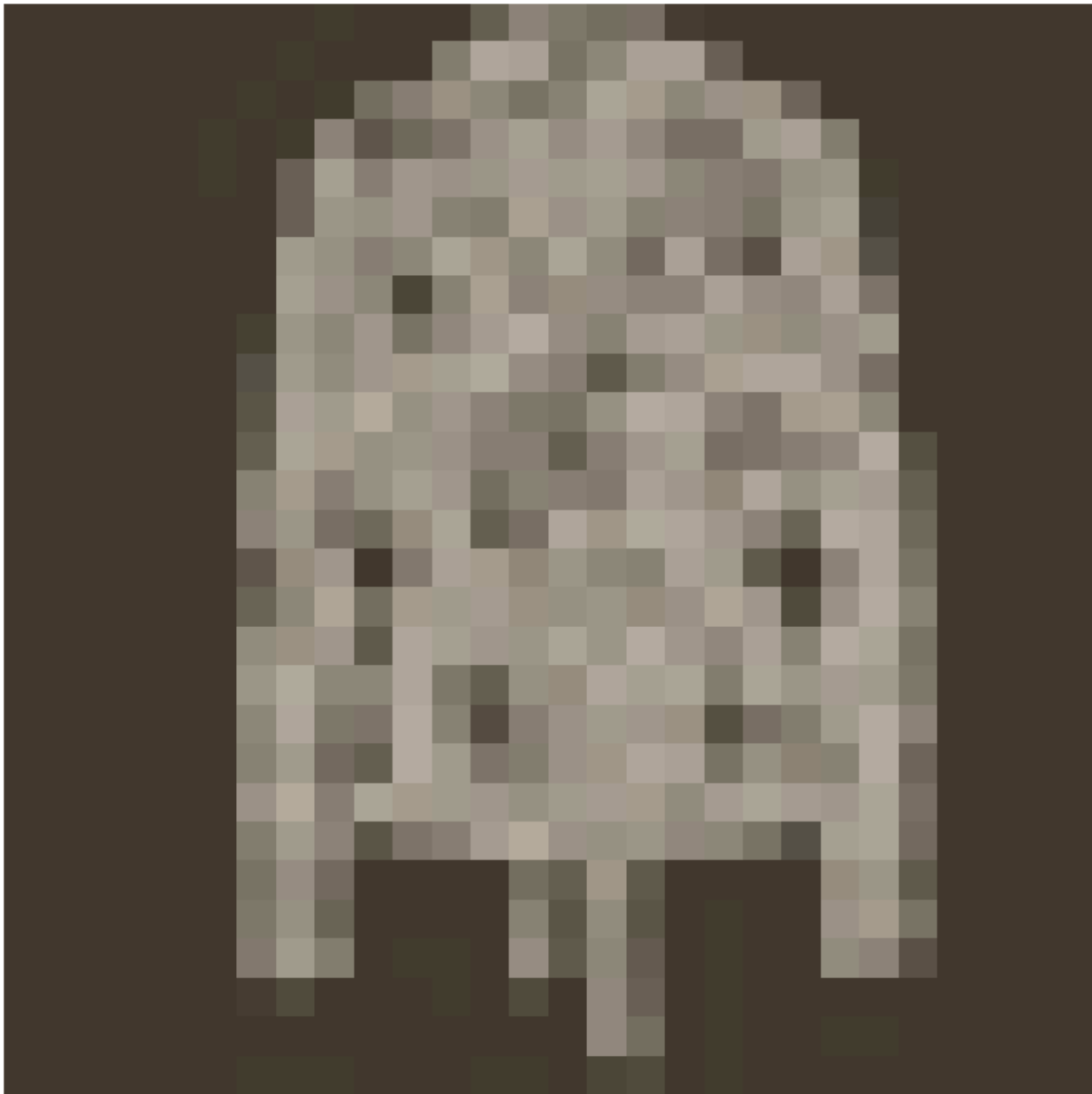
In [2]: # Define a transform to normalize the data
transform = transforms.Compose([transforms.ToTensor(),
                               transforms.Normalize((0.5,), (0.5,))])

# Download and load the training data
trainset = datasets.FashionMNIST('~/.pytorch/F_MNIST_data/', download=True, train=True, transform=transform)
trainloader = torch.utils.data.DataLoader(trainset, batch_size=64, shuffle=True)

# Download and load the test data
testset = datasets.FashionMNIST('~/.pytorch/F_MNIST_data/', download=True, train=False, transform=transform)
testloader = torch.utils.data.DataLoader(testset, batch_size=64, shuffle=True)
```

Here we can see one of the images.

```
In [4]: image, label = next(iter(trainloader))
helper.imshow(image[0,:]);
```



Train a network

To make things more concise here, I moved the model architecture and training code from the last part to a file called `fc_model`. Importing this, we can easily create a fully-connected network with `fc_model.Network`, and train the network using `fc_model.train`. I'll use this model (once it's trained) to demonstrate how we can save and load models.

```
In [5]: # Create the network, define the criterion and optimizer

model = fc_model.Network(784, 10, [512, 256, 128])
criterion = nn.NLLLoss()
optimizer = optim.Adam(model.parameters(), lr=0.001)

In [6]: fc_model.train(model, trainloader, testloader, criterion, optimizer, epochs=2)
```

```
Epoch: 1/2..   Training Loss: 1.737..   Test Loss: 1.010..   Test Accuracy: 0.591
Epoch: 1/2..   Training Loss: 1.010..   Test Loss: 0.766..   Test Accuracy: 0.681
Epoch: 1/2..   Training Loss: 0.879..   Test Loss: 0.674..   Test Accuracy: 0.746
Epoch: 1/2..   Training Loss: 0.839..   Test Loss: 0.654..   Test Accuracy: 0.756
Epoch: 1/2..   Training Loss: 0.801..   Test Loss: 0.648..   Test Accuracy: 0.755
Epoch: 1/2..   Training Loss: 0.702..   Test Loss: 0.593..   Test Accuracy: 0.776
Epoch: 1/2..   Training Loss: 0.726..   Test Loss: 0.621..   Test Accuracy: 0.764
Epoch: 1/2..   Training Loss: 0.678..   Test Loss: 0.561..   Test Accuracy: 0.790
Epoch: 1/2..   Training Loss: 0.690..   Test Loss: 0.576..   Test Accuracy: 0.787
Epoch: 1/2..   Training Loss: 0.648..   Test Loss: 0.557..   Test Accuracy: 0.790
Epoch: 1/2..   Training Loss: 0.659..   Test Loss: 0.558..   Test Accuracy: 0.799
Epoch: 1/2..   Training Loss: 0.601..   Test Loss: 0.532..   Test Accuracy: 0.799
Epoch: 1/2..   Training Loss: 0.627..   Test Loss: 0.525..   Test Accuracy: 0.808
Epoch: 1/2..   Training Loss: 0.568..   Test Loss: 0.505..   Test Accuracy: 0.811
Epoch: 1/2..   Training Loss: 0.628..   Test Loss: 0.538..   Test Accuracy: 0.797
Epoch: 1/2..   Training Loss: 0.657..   Test Loss: 0.529..   Test Accuracy: 0.806
Epoch: 1/2..   Training Loss: 0.592..   Test Loss: 0.521..   Test Accuracy: 0.811
Epoch: 1/2..   Training Loss: 0.609..   Test Loss: 0.522..   Test Accuracy: 0.808
Epoch: 1/2..   Training Loss: 0.599..   Test Loss: 0.495..   Test Accuracy: 0.819
Epoch: 1/2..   Training Loss: 0.578..   Test Loss: 0.483..   Test Accuracy: 0.825
Epoch: 1/2..   Training Loss: 0.551..   Test Loss: 0.494..   Test Accuracy: 0.821
Epoch: 1/2..   Training Loss: 0.574..   Test Loss: 0.481..   Test Accuracy: 0.825
Epoch: 1/2..   Training Loss: 0.553..   Test Loss: 0.492..   Test Accuracy: 0.820
Epoch: 2/2..   Training Loss: 0.564..   Test Loss: 0.505..   Test Accuracy: 0.822
Epoch: 2/2..   Training Loss: 0.567..   Test Loss: 0.486..   Test Accuracy: 0.819
Epoch: 2/2..   Training Loss: 0.525..   Test Loss: 0.479..   Test Accuracy: 0.821
Epoch: 2/2..   Training Loss: 0.546..   Test Loss: 0.490..   Test Accuracy: 0.822
Epoch: 2/2..   Training Loss: 0.589..   Test Loss: 0.480..   Test Accuracy: 0.827
Epoch: 2/2..   Training Loss: 0.544..   Test Loss: 0.475..   Test Accuracy: 0.821
Epoch: 2/2..   Training Loss: 0.545..   Test Loss: 0.479..   Test Accuracy: 0.823
Epoch: 2/2..   Training Loss: 0.579..   Test Loss: 0.482..   Test Accuracy: 0.821
Epoch: 2/2..   Training Loss: 0.529..   Test Loss: 0.463..   Test Accuracy: 0.831
Epoch: 2/2..   Training Loss: 0.523..   Test Loss: 0.480..   Test Accuracy: 0.823
Epoch: 2/2..   Training Loss: 0.558..   Test Loss: 0.484..   Test Accuracy: 0.815
Epoch: 2/2..   Training Loss: 0.556..   Test Loss: 0.460..   Test Accuracy: 0.825
Epoch: 2/2..   Training Loss: 0.552..   Test Loss: 0.447..   Test Accuracy: 0.837
Epoch: 2/2..   Training Loss: 0.530..   Test Loss: 0.455..   Test Accuracy: 0.837
Epoch: 2/2..   Training Loss: 0.501..   Test Loss: 0.470..   Test Accuracy: 0.835
Epoch: 2/2..   Training Loss: 0.516..   Test Loss: 0.469..   Test Accuracy: 0.830
Epoch: 2/2..   Training Loss: 0.522..   Test Loss: 0.464..   Test Accuracy: 0.834
Epoch: 2/2..   Training Loss: 0.538..   Test Loss: 0.455..   Test Accuracy: 0.832
Epoch: 2/2..   Training Loss: 0.505..   Test Loss: 0.460..   Test Accuracy: 0.834
Epoch: 2/2..   Training Loss: 0.482..   Test Loss: 0.446..   Test Accuracy: 0.842
Epoch: 2/2..   Training Loss: 0.492..   Test Loss: 0.436..   Test Accuracy: 0.845
Epoch: 2/2..   Training Loss: 0.507..   Test Loss: 0.445..   Test Accuracy: 0.836
Epoch: 2/2..   Training Loss: 0.528..   Test Loss: 0.456..   Test Accuracy: 0.833
```

Saving and loading networks

As you can imagine, it's impractical to train a network every time you need to use it. Instead, we can save trained networks then load them later to train more or use them for predictions.

The parameters for PyTorch networks are stored in a model's `state_dict`. We can see the state dict contains the weight and bias matrices for each of our layers.

```
In [7]: print("Our model: \n\n", model, '\n\n')
print("The state dict keys: \n\n", model.state_dict().keys())
```

```
Our model:

Network(
  (hidden_layers): ModuleList(
    (0): Linear(in_features=784, out_features=512, bias=True)
    (1): Linear(in_features=512, out_features=256, bias=True)
    (2): Linear(in_features=256, out_features=128, bias=True)
  )
  (output): Linear(in_features=128, out_features=10, bias=True)
  (dropout): Dropout(p=0.5, inplace=False)
)
```

The state dict keys:

```
odict_keys(['hidden_layers.0.weight', 'hidden_layers.0.bias', 'hidden_layers.1.weight', 'hidden_layers.1.bias', 'hidden_layers.2.weight', 'hidden_layers.2.bias', 'output.weight', 'output.bias'])
```

The simplest thing to do is simply save the state dict with `torch.save`. For example, we can save it to a file `'checkpoint.pth'`.

```
In [8]: torch.save(model.state_dict(), 'checkpoint.pth')
```

Then we can load the state dict with `torch.load`.

```
In [9]: state_dict = torch.load('checkpoint.pth')
print(state_dict.keys())

odict_keys(['hidden_layers.0.weight', 'hidden_layers.0.bias', 'hidden_layers.1.weight', 'hidden_layers.1.bias', 'hidden_layers.2.weight', 'hidden_layers.2.bias', 'output.weight', 'output.bias'])

And to load the state dict in to the network, you do model.load_state_dict(state_dict).
```

```
In [10]: model.load_state_dict(state_dict)
```

Out[10]: <All keys matched successfully>

Seems pretty straightforward, but as usual it's a bit more complicated. Loading the state dict works only if the model architecture is exactly the same as the checkpoint architecture. If I create a model with a different architecture, this fails.

```
In [11]: # Try this
model = fc_model.Network(784, 10, [400, 200, 100])
# This will throw an error because the tensor sizes are wrong!
model.load_state_dict(state_dict)
```

```
-----
RuntimeError                                Traceback (most recent call last)
Cell In[11], line 4
      2 model = fc_model.Network(784, 10, [400, 200, 100])
      3 # This will throw an error because the tensor sizes are wrong!
----> 4 model.load_state_dict(state_dict)

File c:\Users\eraye\AppData\Local\Programs\Python\Python313\Lib\site-packages\torch\nn\modules\module.py:2581, in Module.load_state_dict(self, state_dict, strict, assign)
    2573         error_msgs.insert(
    2574             0,
    2575             "Missing key(s) in state_dict: {}".format(
    2576                 ", ".join(f'"{k}"' for k in missing_keys)
    2577             ),
    2578         )
    2580 if len(error_msgs) > 0:
-> 2581     raise RuntimeError(
    2582         "Error(s) in loading state_dict for {}:{}\t{}".format(
    2583             self.__class__.__name__, "\n\t".join(error_msgs)
    2584         )
    2585     )
    2586 return _IncompatibleKeys(missing_keys, unexpected_keys)

RuntimeError: Error(s) in loading state_dict for Network:
  size mismatch for hidden_layers.0.weight: copying a param with shape torch.Size([512, 784]) from checkpoint, the shape in current model is torch.Size([400, 784]).
  size mismatch for hidden_layers.0.bias: copying a param with shape torch.Size([512]) from checkpoint, the shape in current model is torch.Size([400]).
  size mismatch for hidden_layers.1.weight: copying a param with shape torch.Size([256, 512]) from checkpoint, the shape in current model is torch.Size([200, 400]).
  size mismatch for hidden_layers.1.bias: copying a param with shape torch.Size([256]) from checkpoint, the shape in current model is torch.Size([200]).
  size mismatch for hidden_layers.2.weight: copying a param with shape torch.Size([128, 256]) from checkpoint, the shape in current model is torch.Size([100, 200]).
  size mismatch for hidden_layers.2.bias: copying a param with shape torch.Size([128]) from checkpoint, the shape in current model is torch.Size([100]).
  size mismatch for output.weight: copying a param with shape torch.Size([10, 128]) from checkpoint, the shape in current model is torch.Size([10, 100]).
```

This means we need to rebuild the model exactly as it was when trained. Information about the model architecture needs to be saved in the checkpoint, along with the state dict. To do this, you build a dictionary with all the information you need to completely rebuild the model.

```
In [12]: checkpoint = {'input_size': 784,
                    'output_size': 10,
                    'hidden_layers': [each.out_features for each in model.hidden_layers],
                    'state_dict': model.state_dict()}

torch.save(checkpoint, 'checkpoint.pth')
```

Now the checkpoint has all the necessary information to rebuild the trained model. You can easily make that a function if you want. Similarly, we can write a function to load checkpoints.

```
In [13]: def load_checkpoint(filepath):
checkpoint = torch.load(filepath)
model = fc_model.Network(checkpoint['input_size'],
                        checkpoint['output_size'],
                        checkpoint['hidden_layers'])
model.load_state_dict(checkpoint['state_dict'])

return model
```

```
In [14]: model = load_checkpoint('checkpoint.pth')
print(model)
```

```
Network(
  (hidden_layers): ModuleList(
    (0): Linear(in_features=784, out_features=400, bias=True)
    (1): Linear(in_features=400, out_features=200, bias=True)
    (2): Linear(in_features=200, out_features=100, bias=True)
  )
  (output): Linear(in_features=100, out_features=10, bias=True)
  (dropout): Dropout(p=0.5, inplace=False)
)
```